



ÉCOLE SUPÉRIEURE POLYTECHNIQUE

Université Cheikh Anta Diop de Dakar (UCAD)

Département Génie Informatique

DST-INFO2 — Génie Logiciel et Systèmes d'Information (GLSI)

RAPPORT DE PROJET JAVA

Système de Gestion d'une Bibliothèque Municipale

Réalisé par :

Vieux Dame LEYE

Seydina Mouhamadou Lamine DIOUF

Mouhamadou Mansour Ndiaye

Année académique : 2025 — 2026

1. Introduction

Ce rapport présente le projet Java réalisé dans le cadre du cours de Programmation Orientée Objet (POO) en deuxième année de DST Informatique (GLSI) à l'École Supérieure Polytechnique de Dakar (ESP — UCAD).

Le projet consiste en la conception et l'implémentation d'un système de gestion d'une bibliothèque municipale. L'objectif est de modéliser en Java les différentes entités du système (livres, membres, emprunts, amendes, stock, véhicule, capteurs) en respectant les principes de la Programmation Orientée Objet, notamment l'encapsulation, la validation des données, et la persistance en base de données MySQL.

Le système couvre les fonctionnalités suivantes :

- Gestion du catalogue de livres et des exemplaires physiques
- Inscription et gestion des membres de la bibliothèque
- Gestion des emprunts avec calcul automatique des amendes en cas de retard
- Caisse centralisée pour le traitement des paiements
- Surveillance des conditions de conservation (température, humidité)
- Gestion d'un véhicule pour les livraisons entre annexes
- Persistance complète de toutes les données en base de données MySQL

2. Architecture du Projet

2.1 Structure des packages

Le projet est organisé en trois packages distincts, chacun ayant un rôle précis :

Package	Classes	Rôle
db/	DatabaseManager	Gestion de la connexion MySQL (Singleton de connexion)
model/	Livre, Membre, Exemplaire, Stock, Capteur, Vehicule	Entités métier représentant les objets du domaine
service/	Emprunt, Amende, Caisse	Services métier gérant les opérations complexes

2.2 Organisation des fichiers

bibliotheque-municipale/

```

├── src/
│   ├── db/DatabaseManager.java
│   ├── model/Livre.java, Membre.java, Exemplaire.java
│   ├── model/Stock.java, Capteur.java, Vehicule.java
│   ├── service/Emprunt.java, Amende.java, Caisse.java
│   └── Main.java
├── lib/mysql-connector-j-9.6.0.jar
├── diagramme_UML.png
├── rapport_bibliotheque_municipale.pdf
├── README.md
└── schema.sql

```

3. Description des Classes

3.1 Classe DatabaseManager (package db)

La classe DatabaseManager est le point d'entrée unique vers la base de données MySQL. Elle est implémentée comme une classe utilitaire statique (aucune instanciation) et joue le rôle de gestionnaire centralisé de la connexion.

Caractéristiques principales :

- Connexion unique et réutilisée (pattern Singleton de connexion)
- Chargement automatique du driver MySQL via Class.forName()
- Initialisation automatique du schéma à la première connexion (CREATE TABLE)
- URL de connexion : jdbc:mysql://localhost:3306/bibliotheque

3.2 Classes du package model

3.2.1 Classe Livre

Représente une notice bibliographique dans le catalogue. Un livre est identifié par son ISBN (13 chiffres, validé par expression régulière). Ses attributs sont : id, titre, auteur, isbn, genre, anneePublication (validée entre 1450 et 2100).

3.2.2 Classe Membre

Représente un adhérent de la bibliothèque. La classe impose une limite de 3 emprunts simultanés (constante `MAX_EMPRUNTS = 3`). Elle valide le format de l'email et du numéro de téléphone. Elle expose les méthodes `peutEmprunter()`, `incrémenterEmprunts()` et `decrémenterEmprunts()`.

3.2.3 Classe Exemple

Représente un exemplaire physique d'un livre. Un exemplaire possède un état parmi : `DISPONIBLE`, `EMPRUNTE`, `ABIME` ou `PERDU` (énumération interne). La méthode `marquerEmprunte()` vérifie que l'exemplaire est disponible avant de changer son état.

3.2.4 Classe Stock

Gère la collection d'exemplaires de la bibliothèque en mémoire. Elle offre des méthodes pour ajouter, retirer, rechercher le premier exemplaire disponible pour un livre donné, et compter les disponibles.

3.2.5 Classe Capteur

Surveille les conditions de conservation des livres. Elle définit des seuils constants : température entre 15°C et 22°C, humidité entre 40% et 60%. Chaque capteur est associé à un emplacement physique dans la bibliothèque.

3.2.6 Classe Vehicule

Représente un véhicule utilisé pour les livraisons entre annexes. Son état peut être `DISPONIBLE`, `EN_LIVRAISON` ou `EN_MAINTENANCE`. La méthode `partirEnLivraison()` vérifie la disponibilité avant le départ.

3.3 Classes du package service

3.3.1 Classe Emprunt

Gère le cycle de vie d'un emprunt. La durée maximale est de 14 jours (`DUREE_MAX_JOURS = 14`) et l'amende est de 100 FCFA par jour de retard. La méthode `retourner()` calcule automatiquement le retard et crée une Amende si nécessaire. Elle met à jour l'état de l'exemplaire et le compteur du membre.

3.3.2 Classe Amende

Représente une amende générée suite à un retard. Elle contient le nombre de jours de retard, le montant en FCFA, et la date de paiement. La méthode `marquerPayee()` est à visibilité package (accès réservé à la Caisse).

3.3.3 Classe Caisse

Implémente le patron de conception Singleton — une seule instance existe dans tout le programme. Elle gère les listes d'amendes en attente et payées, le solde en FCFA, et expose les méthodes

enregistrerAmende(), payerAmende() et retirer(). Le solde est persisté en base de données via saveSolde().

4. Persistance des Données

4.1 Schéma de la base de données

La base de données MySQL nommée 'bibliotheque' contient les tables suivantes :

Table	Colonnes principales
livre	id, titre, auteur, isbn (UNIQUE), genre, annee_publication
membre	id, nom, prenom, email (UNIQUE), telephone, date_inscription
exemplaire	id, livre_id (FK), etat (ENUM), numero_rayon
emprunt	id, membre_id (FK), exemplaire_id (FK), date_emprunt, date_retour_prevue, date_retour_effective, est_clos
amende	id, emprunt_id (FK, UNIQUE), jours_retard, montant, est_payee, date_paiement
caisse	id (DEFAULT 1), solde — une seule ligne (Singleton)
capteur	id, emplacement, temperature, humidite, derniere_lecture, actif
vehicule	id, immatriculation (UNIQUE), modele, etat (ENUM), date_derniere_revision, annexe_actuelle

4.2 Méthodes CRUD

Chaque classe persistée dispose de trois méthodes CRUD standardisées :

- `save()` : INSERT si `id == 0` (nouvelle entité), UPDATE sinon
- `findById(int id)` : SELECT par clé primaire, retourne null si non trouvé
- `delete()` : DELETE par clé primaire

5. Patrons de Conception Utilisés

Patron	Classe concernée	Description
Singleton	Caisse	Une seule instance via <code>getInstance()</code> , constructeur privé

Patron	Classe concernée	Description
Utilitaire statique	DatabaseManager	Connexion unique partagée par toutes les classes
Encapsulation	Toutes les classes	Attributs privés, setters avec validation, exceptions levées
Énumération	Exemplaire, Vehicule	États finis typés (DISPONIBLE, EMPRUNTE, EN_LIVRAISON...)

6. Scénario de Démonstration (Main)

La classe Main simule un scénario complet et réaliste en 5 sections, avec affichage de l'état de la base de données après chaque action significative.

Section 1 — Création des entités

- Création de 2 livres (littérature africaine : Mariama Bâ, Camara Laye)
- Création de 3 exemplaires répartis sur différents rayons
- Inscription de 2 membres avec coordonnées valides
- Initialisation d'un capteur et d'un véhicule
- Simulation d'une alerte capteur (humidité hors seuil : 72%)
- Simulation d'une livraison entre annexes

Section 2 — Gestion du Stock

- Affichage de l'état complet du stock
- Comptage des exemplaires disponibles par livre

Section 3 — Scénario d'emprunt

- Emprunt normal → retour dans les délais → aucune amende
- Emprunt avec retard simulé (20 jours) → amende de 600 FCFA générée
- Enregistrement de l'amende dans la Caisse → paiement → mise à jour du solde

Section 4 — Tests de validation

- ISBN invalide (5 chiffres) → IllegalArgumentException capturée
- Année invalide (2500) → IllegalArgumentException capturée
- Email vide → IllegalArgumentException capturée
- Paiement d'une amende null → IllegalArgumentException capturée

- Retrait d'un exemplaire du stock

Section 5 — État final

- Affichage du nombre d'entités en base (livres, membres, emprunts, amendes...)
- Affichage de l'état de la caisse (solde, amendes en attente, amendes payées)

7. Compilation et Exécution

7.1 Prérequis

- JDK 21 LTS installé
- XAMPP avec MySQL démarré
- Base de données 'bibliotheque' créée dans phpMyAdmin
- Fichier mysql-connector-j-9.6.0.jar présent dans le dossier lib/

7.2 Commandes

Compilation (depuis la racine du projet) :

```
javac -cp "lib\mysql-connector-j-9.6.0.jar" -d out src\db\DatabaseManager.java src\model\*.java src\service\*.java src>Main.java
```

Exécution :

```
java -cp "out;lib\mysql-connector-j-9.6.0.jar" Main
```


8. Conclusion

Ce projet nous a permis de mettre en pratique les principes fondamentaux de la Programmation Orientée Objet en Java dans un contexte applicatif concret. Nous avons conçu un système complet comportant 9 classes réparties en 3 packages, avec une persistance intégrale des données en base de données MySQL.

Les principaux acquis de ce projet sont : la maîtrise de l'encapsulation et de la validation des données, l'utilisation de patrons de conception (Singleton, utilitaire statique), la gestion des exceptions, la connexion Java-MySQL avec JDBC, et la collaboration en équipe via Git et GitHub.